

12. Eventual Consistency & Scaling Out: Key-Value Storage

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2026

12.1 Vorschau

- Amazon Dynamo Key-Value Storage
- DXRAM

12.2 Allgemeines

- Key-Value Store: speichert Key-Value Tupel (Schlüssel-Wert-Paare)
 - Datenverteilung auf die Server i.d.R. mithilfe von Consistent-Hashing
 - In der Regel alle Server untereinander vernetzt → Lookup in $O(1)$
 - Je nach System mit Replikation oder Checkpointing
 - Die meisten KV-Stores laufen nur innerhalb eines Data-Centers
- Es gibt sehr viele Key-Value Stores; Beispiele:
 - Redis
 - DXRAM
 - Couchbase
 - Amazon Dynamo

12.3 Amazon Dynamo

12.3 Amazon Dynamo

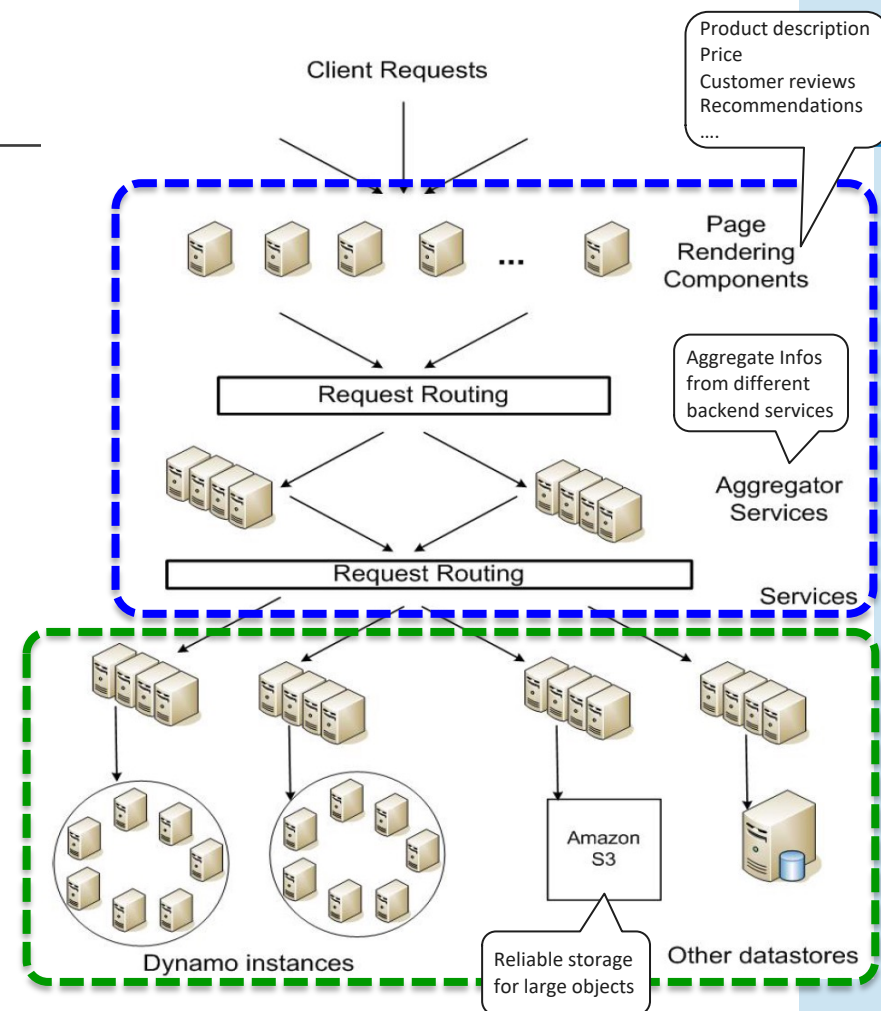
- Verteilter Key-Value Storage für Zustände von Diensten (stateful services)
 - Verteilt über mehrere Data-Centers
- Verwendet unter anderem für das Online-Shopping
- 10^6 Server in über 80 Data Centers
- Oft mehr als 10^7 gleichzeitige Kunden
- (Zahlen sind von 2007)



aws = amazon web services

Amazon Stateful Services

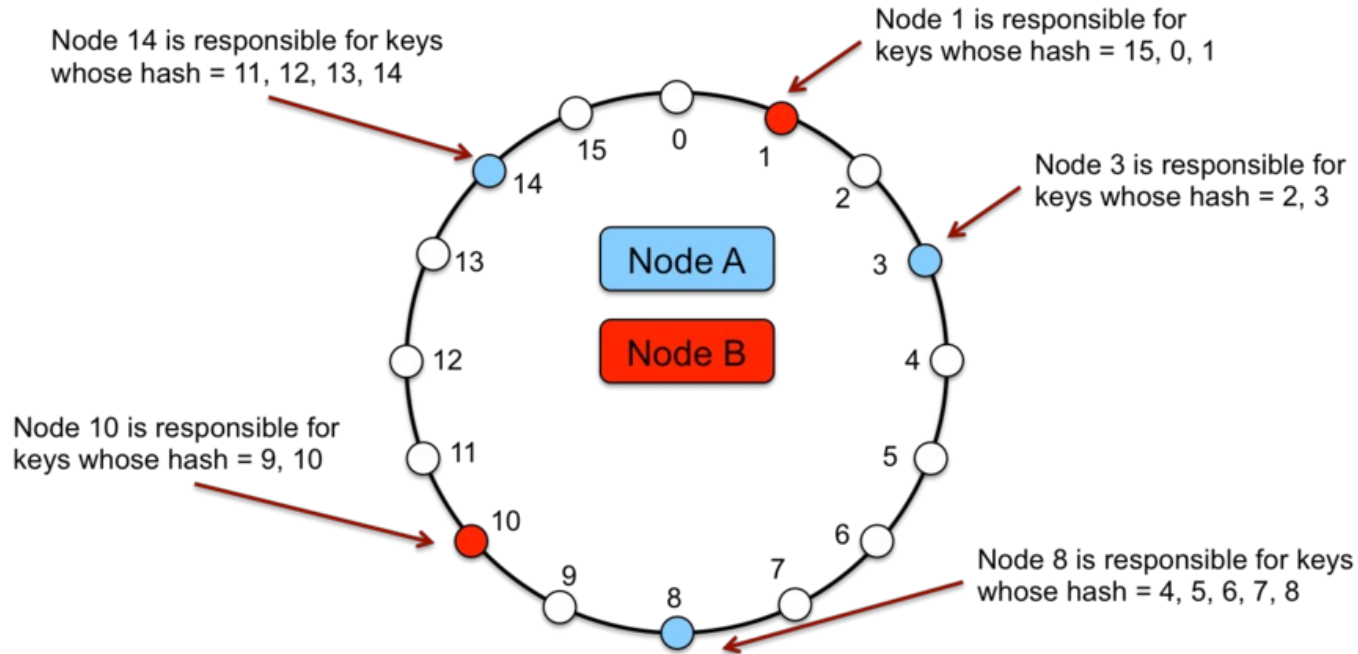
- Service-Orientierte Architektur →
 - Stateless Web servers & Aggregators
 - Stateful Storage Servers
- Service-Level Agreement (SLA)
 - Beispiel: garantierte Antwortzeit innerhalb von 300ms für 99.9% aller Anfragen



Datenverteilung

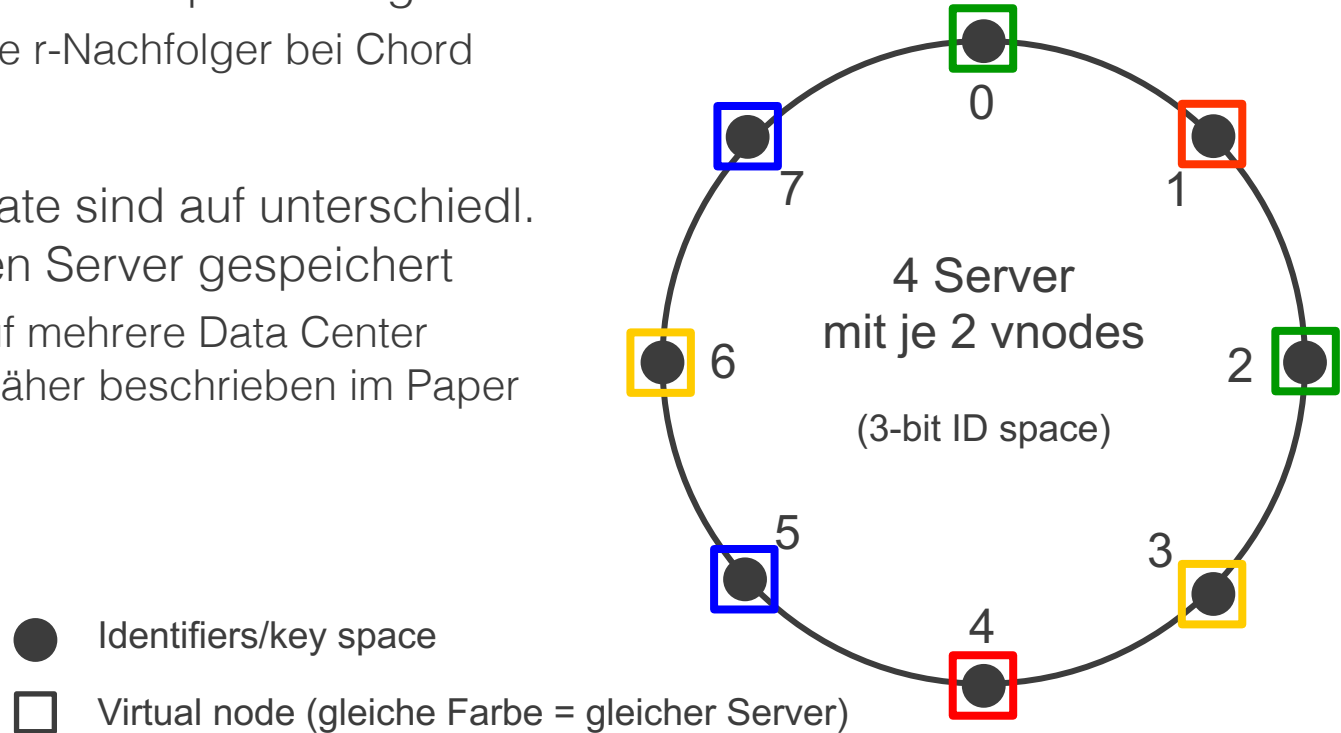
- Verwendet eine DHT ähnlich wie bei Chord
 - Als Hash-Funktion wird MD5 (128 Bit) verwendet
- Annahmen:
 - Wenig Churn innerhalb eines Data-Centers
 - Pro Dynamo-Instanz max. ca. 12.000 Server
- Vollvermaschte Vernetzung für alle Server der DHT $\rightarrow O(1)$ für alle Anfragen

Lastverteilung mithilfe virtueller Knoten (vnodes)



Replikation

- Replikation von KV-Tupeln erfolgt auf M vnodes
 - Vergleiche r-Nachfolger bei Chord
- Alle M Replikate sind auf unterschiedl. physikalischen Server gespeichert
 - verteilt auf mehrere Data Center
→ nicht näher beschrieben im Paper



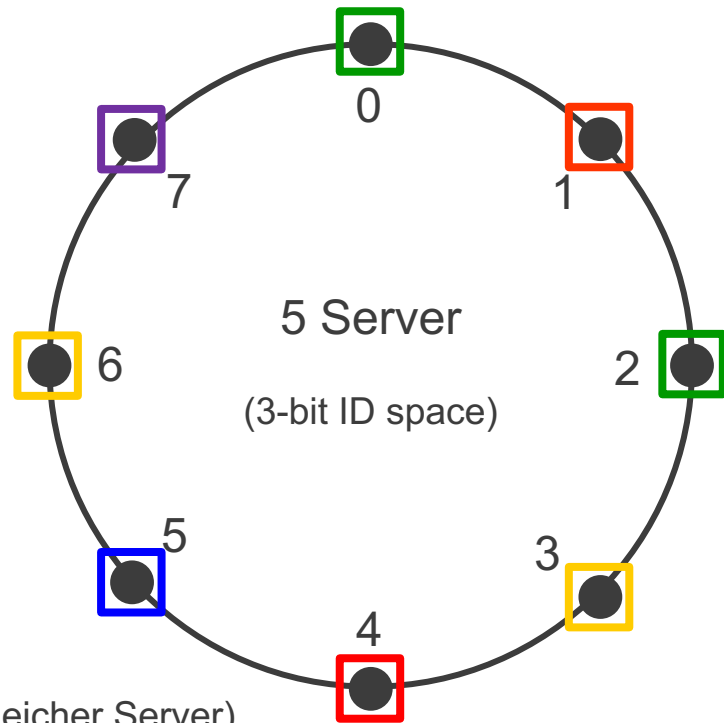
Replikation – Preference-List & Koordinator

$M = 4$ (Anzahl der Replikate)

Key 0: **Preference-List** für die Replikate z.B.
vnodes: $\{0, 1, 3, 5\} \rightarrow \{\text{gruen}, \text{rot}, \text{gelb}, \text{blau}\}$
gruen ist der **Koordinator-Server** für Key 0

Das Replikat, welches einen Write bekommt wird diesen Write zum Koordinator. Koordinator ist im fehlerfreien Fall immer der erste Eintrag in der Preference-List.

- Identifiers/key space
- Virtual node (gleiche Farbe = gleicher Server)



Lese-/Schreib-Anfragen

- Werden vom Koordinator verarbeitet und an die ersten N Server (inkl. Koordinator) in der Preference-List geschickt ($N < M$)
 - M = Anzahl der Replikate
- Wichtig: ein Schreibzugriff speichert eine neue Version des Key-Value-Tupels und überschreibt nicht eine bestehende Version → [Multi-Version Data-Store](#)
- Es wird ein quorum-basiertes Protokoll verwendet: $W + R > N$
 - Schreib-Quorum: Schreib Anfrage ist erfolgreich, wenn W von N Servern bestätigen
 - Lese-Quorum: Leseanfrage ist erfolgreich, wenn R von N Servern antworten

Quorum-Parameter: W, R und N

- N definiert die Dauerhaftigkeit der Daten (Default $N=3$)
- W und R legen den **Kompromiss zwischen Verfügbarkeit und Konsistenz fest**
 - $W = 1$ ($R = 3$): schnelles Schreiben, schwache Dauerhaftigkeit, langsames Lesen
 - $R = 1$ ($W = 3$): langsames Schreiben, gute Dauerhaftigkeit, schnelles Lesen
- Wieso $W + R > N$?
 - Lese- und Schreibquorum überlappen, sofern keine Fehler vorhanden sind
 - Damit sehen Lesezugriffe im fehlerfreien Fall immer die letzte Aktualisierung

Erholung von temporären Fehlern

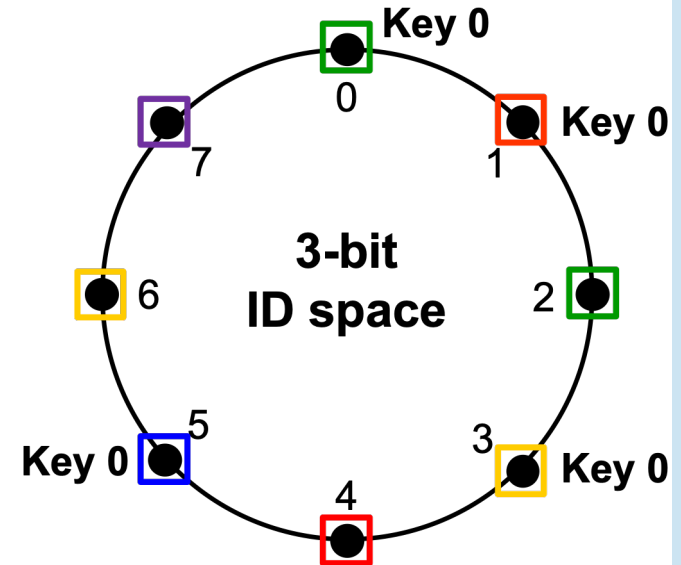
- Sloopy¹-Quorum:
 - Fehlerfreier Fall: es werden immer die ersten N Server der Preference-List verwendet
 - Fehlerfall: es werden ggf. andere Server in der Preference-List verwendet, falls welche am Anfang ausfallen
- Konsequenzen
 - Es muss nicht gewartet werden, bis ausgefallene Server wiederhergestellt sind
 - Aber es können Konflikte zwischen verschiedenen Versionen entstehen
 - Wieder ein **Kompromiss zwischen Verfügbarkeit und Konsistenz**

¹ dt. schlampig

Sloppy-Quorum + Hinted-Handoff

- Key 0: Preference-List {gruen, rot, gelb, blau}, $M=4$
- $N = 3$: {gruen, rot, gelb}; kein Fehler

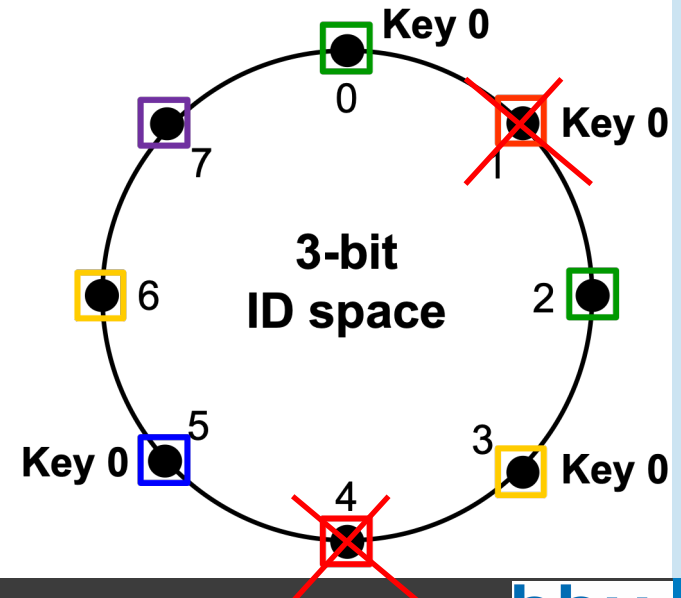
● Identifiers/key space
□ Virtual node:
5 colors → 5 physical nodes



Sloppy-Quorum + Hinted-Handoff

- Key 0: Preference-List {gruen, rot, gelb, blau}, M=4
- N = 3: {gruen, rot, gelb}; kein Fehler
- Falls nun rot ausfällt, so gehen die Anfragen zu {gruen, gelb, blau}
- Hinted-Handoff
 - blau bedient vorerst Anfragen
 - Wird aber vom Koordinator informiert, dass rot zuständig ist → „hinted handoff“
 - blau schickt sein Replikat zu rot, wenn dieser Server wieder online ist

- Identifiers/key space
- Virtual node:
5 colors → 5 physical nodes

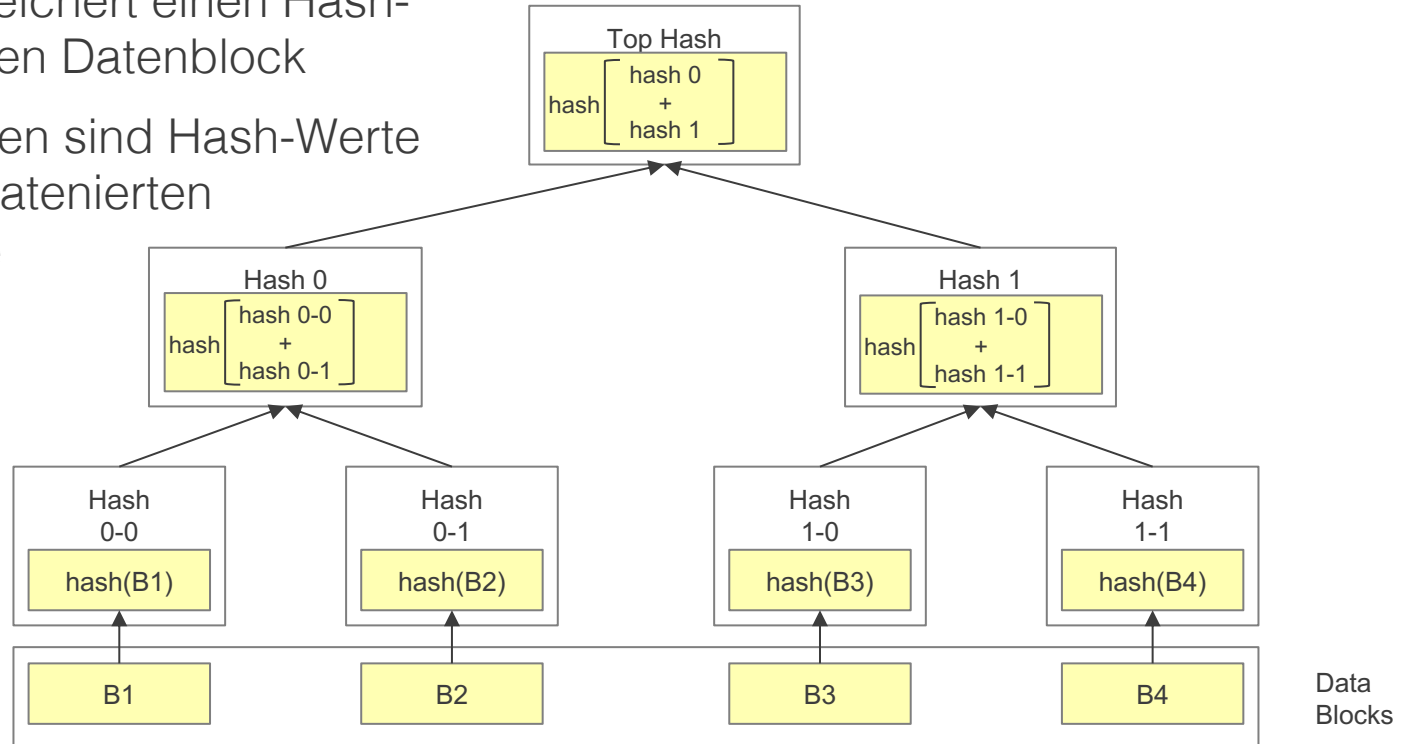


Anti-Entropy-Protokoll (Replica-Synchronisierung)

- Ein Anti-Entropy-Protokoll ist ein Verfahren zur nachträglichen Synchronisation von Replikaten in einem verteilten System.
- Hier notwendig zur Behandlung von Versionskonflikten in Replikaten nach Partitionierungen und permanenten Serverausfällen
- **Zunächst müssen Inkonsistenzen erkannt werden**
 - **erfolgt effizient mithilfe von Merkle-Trees**
 - Ein Baum für jeden virtuellen Server (und damit pro Key-Range)
 - Erlaubt es **große Datenmengen effizient auf Unterschiede zu überprüfen**, ohne alle Daten übertragen zu müssen

Beispiel: binärer Merkle-Tree (nicht in Dynamo verwendet)

- Ein Blatt speichert einen Hash-Wert für einen Datenblock
- Innere Knoten sind Hash-Werte für die konkatenierten Hash-Werte der Kinder



Anti-Entropy-Protokoll (Replica-Synchronisierung)

- **Merkle-Tree** erlaubt es Inkonsistenzen schnell hierarchisch zu erkennen
 - Sind die Wurzeln identisch, besteht kein Problem
 - Andernfalls wird top-down das Problem eingegrenzt
 - Da wir eine stabile Umgebung haben wird es selten Probleme geben, daher werden i.d.R. nur wenig Daten über das Netzwerk ausgetauscht

Auflösen von Inkonsistenzen

- Mithilfe der Merke-Trees können Konflikte nur erkannt werden
- Zusätzlich werden Versionsnummern verwendet, um diese aufzulösen
- Ziel: neuere Versionen überschreiben ältere Versionen
- Das Ganze soll ohne zentralem Koordinator, also dezentral, funktionieren
- Als Versionsnummer wird eine Vektorzeit verwendet

Vektorzeit in Dynamo

- Jedes KV-Tupel wird mit einem Vektorzeitstempel gespeichert
 - Die Implementierung weicht von der Original-Vektorzeit ab
- Die Anzahl der Elemente im Vektor soll minimal sein
 - Es gibt ein Element pro Server, über den ein Update erfolgt ist
 - Wdlg.: Updates erfolgen immer über den Koordinator
 - Normalerweise ist immer der erste Server in der Preference-List der Koordinator
 - Im fehlerfreien Fall gibt also nur ein Element in einem Vektor
 - Sobald ein Update über einen anderen Server/Koordinator erfolgt wird ein zusätzliches Element angelegt
 - Dies tritt nur im Fehlerfall ein, wenn der Koordinator gewechselt werden muss

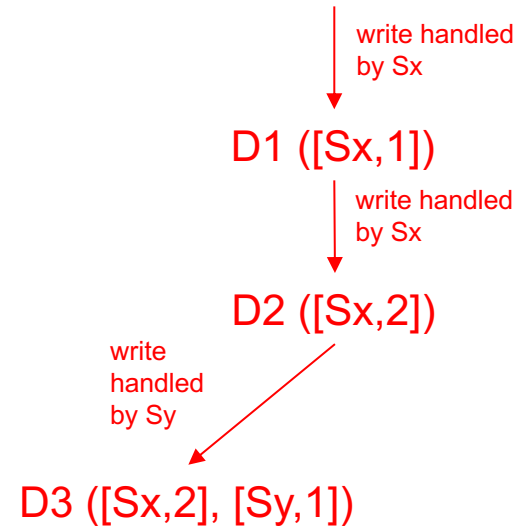
Vektorzeit in Dynamo

- Zusätzlich wird die Anzahl der Elemente in einem Vektor auf 10 begrenzt
 - Notfalls wird das älteste Element verworfen, wodurch manche Inkonsistenzen nicht mehr erkannt werden können
 - Es ist jedoch sehr unwahrscheinlich, dass so viele Fehler auftreten, sodass mehr als 10 Elemente benötigt würden
 - Dies würde nur eintreten, wenn ständig Server nicht erreichbar sind und dadurch ständig neue Koordinatoren verwendet werden müssen

Beispiel: Vektorzeit in Dynamo

→ client1

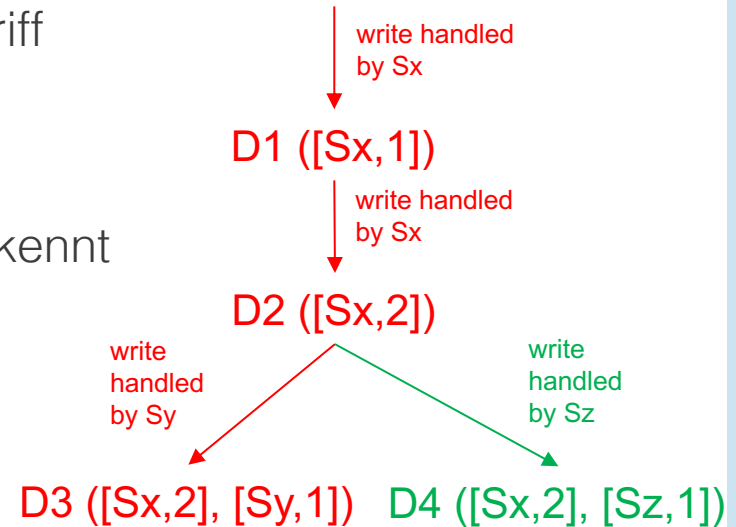
- Client 1 schreibt ein neues Tupel mit Key=D (Version 1). Der Zugriff läuft über den Server Sx
- Später aktualisiert Client 1 das Tupel D. Geht wieder über den gleichen Server Sx → System speichert nun D2
- Anschließend aktualisiert Client 1 das Tupel D, was aufgrund eines Fehlers nun über den Server Sy passiert → System speichert nun D3



Beispiel: Vektorzeit in Dynamo (2)

→ client1
→ client2

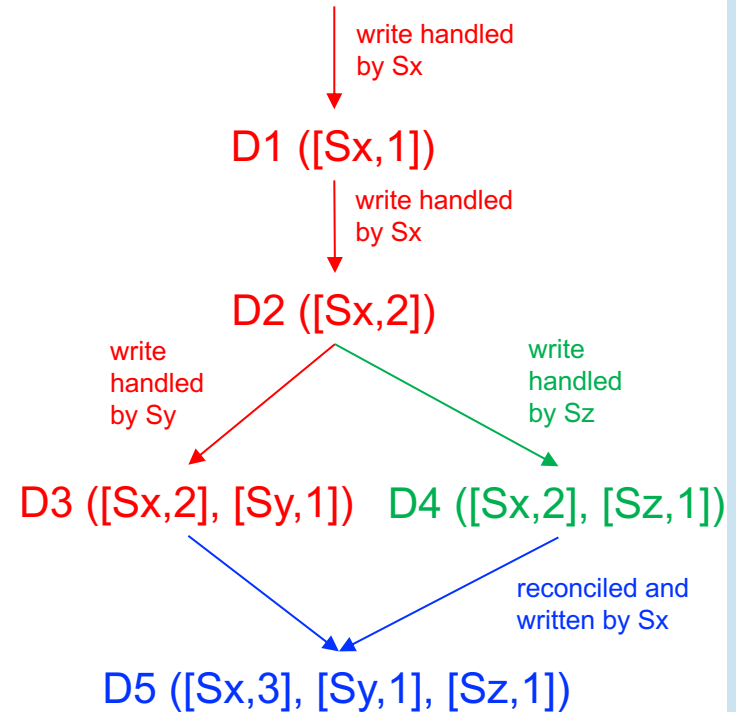
- Client 2 liest Tupel D2 und aktualisiert es dann. Aufgrund eines Fehlers läuft der Schreibzugriff über den Server Sz
- Anhand der Vektorzeitstempel D3 und D4 erkennt Dynamo nebenläufige Aktualisierungen
 - Beide Versionen werden gespeichert
 - Die Anwendung muss diese später zusammenführen



Beispiel: Vektorzeit in Dynamo (3)

→ client1
→ client2
→ client3

- Nun liest ein Client 3 das Tupel und Dynamo liefert D3 und D4 beim Lesezugriff
- Der Client 3 führt nun diese beiden Versionen in D5 zusammen



Typisches Beispiel bei Amazon: Warenkorb

- Es soll immer möglich sein Artikel in den Warenkorb zu legen
- Tritt eine Netzwerkpartitionierung ein, so können ohne Unterbrechung weiter Artikel in den Warenkorb gelegt werden.
- Offensichtlich können hierbei Situationen entstehen, bei denen die Replikate des Warenkorbs in Konflikt stehen, aber einfach zusammengeführt werden können, sobald die Netzwerkpartitionierung aufgehoben ist.
(Es werden einfach alle Artikel behalten)

Zusammenfassung

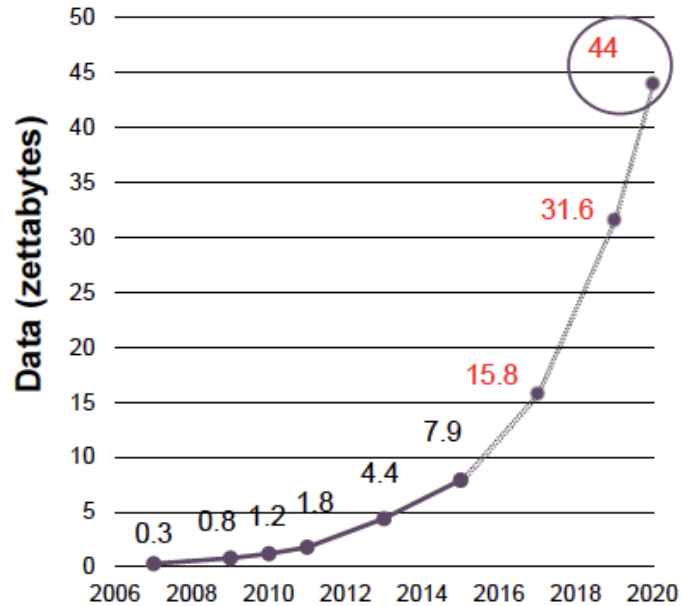
- Vorteile:
 - Hochverfügbar
 - Inkrementell skalierbar
 - Garantiert kleine Zugriffslatenzen (durch eventual consistency)
- Nachteile
 - Skaliert nicht unbegrenzt, bedingt durch voll-vermaschte Vernetzung
 - Konflikte müssen durch die Anwendung aufgelöst werden

- G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, W. Vogels “Dynamo: amazon's highly available key-value store”, ACM SIGOPS, Stevenson, USA, 2007.

12.4 DXRAM

Need answers quickly and on bigger data

Data growth



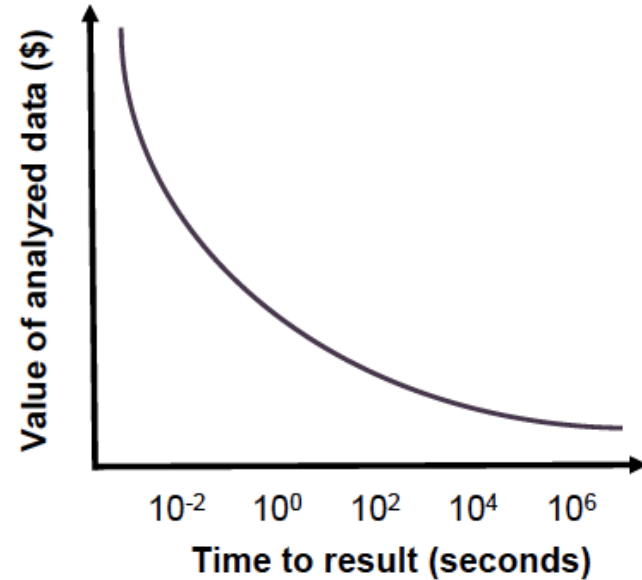
Data nearly doubles every two years (2013-20)

By 2020:

8B people

20B connected devices

100B infrastructure devices



Low Latency Data Access is important for

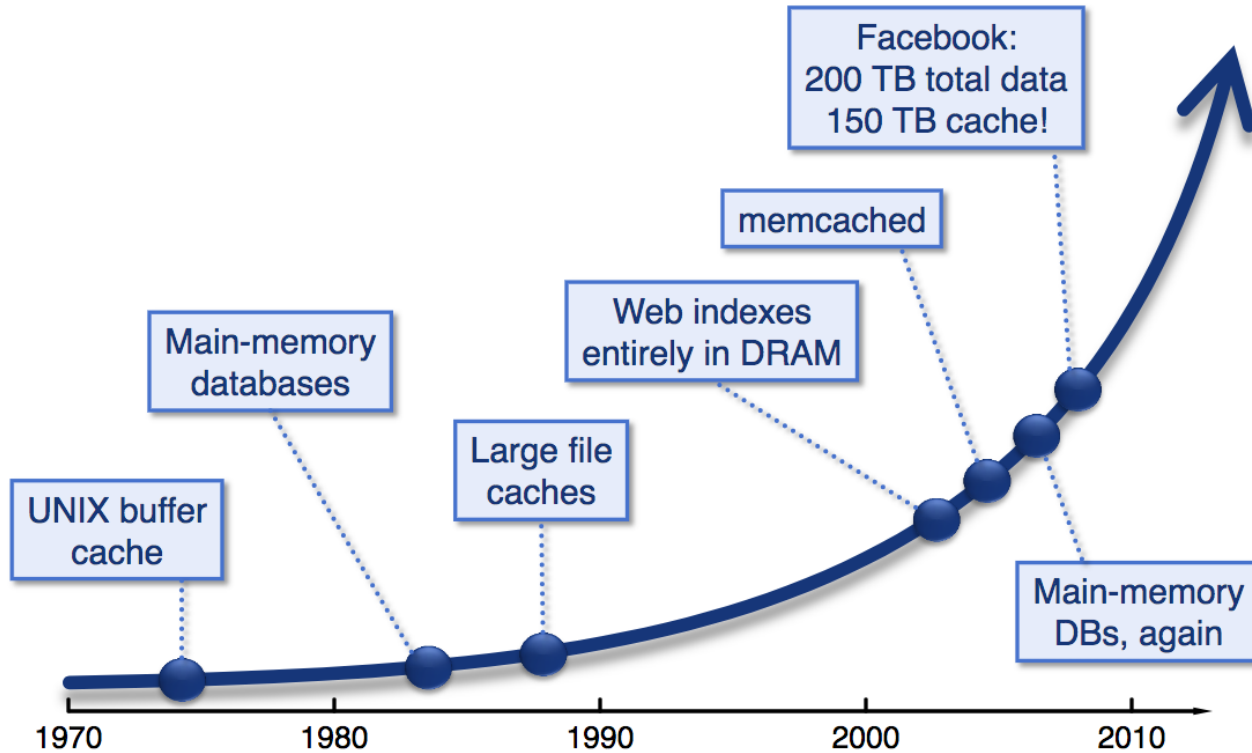
- Interactive applications, e.g. social networks
- Online analytics, e.g. bioinformatics, WWW
- Online state management, e.g. for mobile networks

I/O is the new bottleneck

Memory technology	Access Latency		Max. Throughput	
	Read	Write	Read	Write
Hard disk	9ms	9ms	112 MB/s	45 MB/s
NAND flash	47us	15us	3.0 GB/s	2.6 GB/s
DRAM	70ns	70ns	13 GB/s	13 GB/s
SRAM	1-2ns	1-2ns	>100 GB/s	>100 GB/s

<https://www.extremetech.com/extreme/211087-intel-micron-reveal-xpoint-a-new-memory-architecture-that-claims-to-outclass-both-ddr4-and-nand>

DRAM in Storage Systems



The Case for RAMClouds: Scalable High-Performance Storage Entirely in DRAM, Osterhout et.al.

Caching is not sufficient

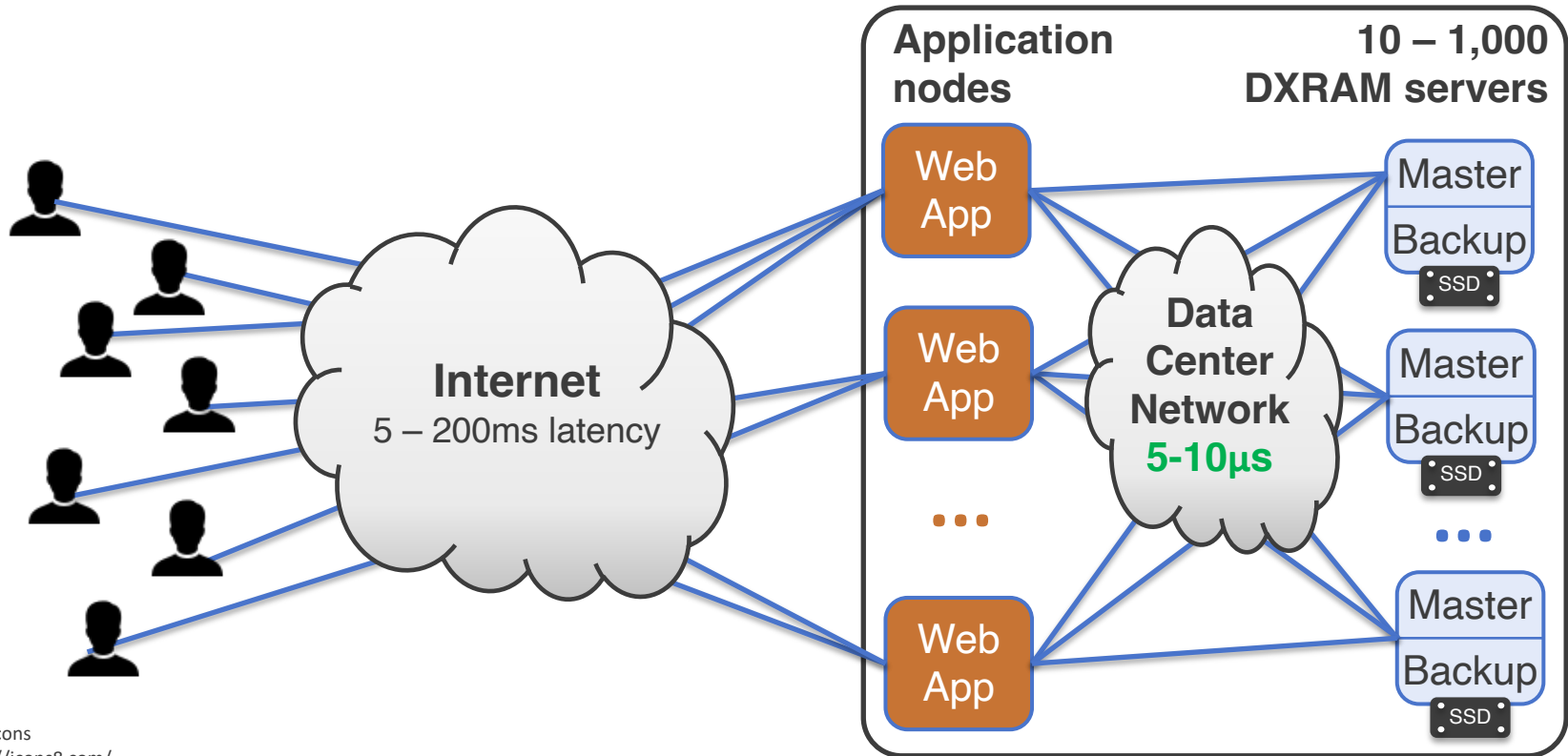
- High cache hit rates needed because of the large access time gap between DRAM and disk
 - Even a 1% miss ratio for a DRAM cache costs a factor of 10x in performance
- Graph applications have **irregular access patterns**
 - **caches need to be very large**
 - Facebook keeps around 75% of all data always in caches
 - Used up to 1.000 Memcached servers for caching
- Caches need to be **manually synchronized** with backend storage to avoid data loss in case of server crashes or power outages

DXRAM: All data always in memory

- Distributed key-value store for data centers.
- Key-value data tuples: “chunks”
 - Key: 64 bit globally unique sequential chunk ID (CID)
- Optimized for handling billions of small chunks.
- **All data always in RAM**
- Persistency through remote logging to raw device (SSD aware).
- Fast parallel recovery of many small chunks
- Based on InfiniBand network 10 – 56 Gbit/s
- Written in Java, open source at Github.



DXRAM: As a fast backend key-value store



User icons
<https://icons8.com/>

DXRAM: Challenges & Approach

- Efficient lookup for billions of small chunks
 - Consistent hashing; fully meshed network → $O(1)$ lookup
- Storage management for billions of small objects
 - Custom allocator → 5 – 10% metadata overhead in RAM.
- Scalability: fast network & P2P concepts
- Reliability: fast recovery
 - A failed server should be recovered in less than 2 seconds.

Fault tolerance

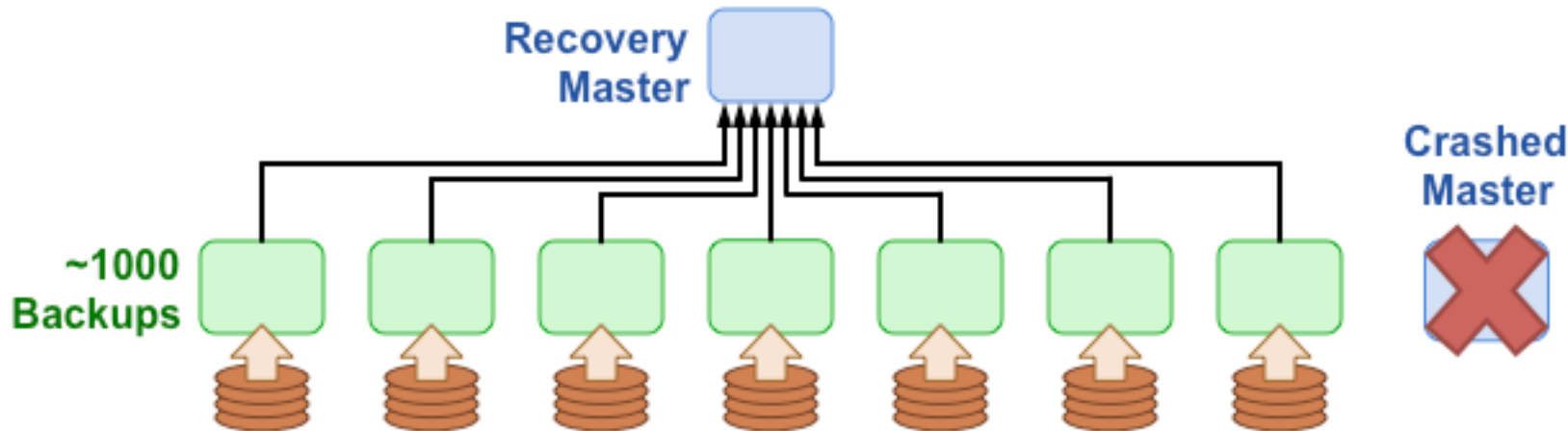
- Crash-recovery model
 - Single or several servers crash (likely)
 - Data center power outage (very seldom)
 - Network may be partitioned (very seldom)
- Data is replicated in logs, stored on **remote** SSDs
 - Remote replication allows immediately recovery
 - Persistence in case of data center power outage
- Fast parallel server recovery (1 - 2 sec.)
 - Minimizing impact for applications
 - While avoiding overhead of in-memory replication

Crash-Recovery: naiv approach

- Replicate in-memory state of one server in one remote backup log
- Problem: disk is too slow
 - E.g. backup state has 64 GB
 - SSD may read 1 GB/s
 - Time to read 64 GB from disk = 64s > 1min

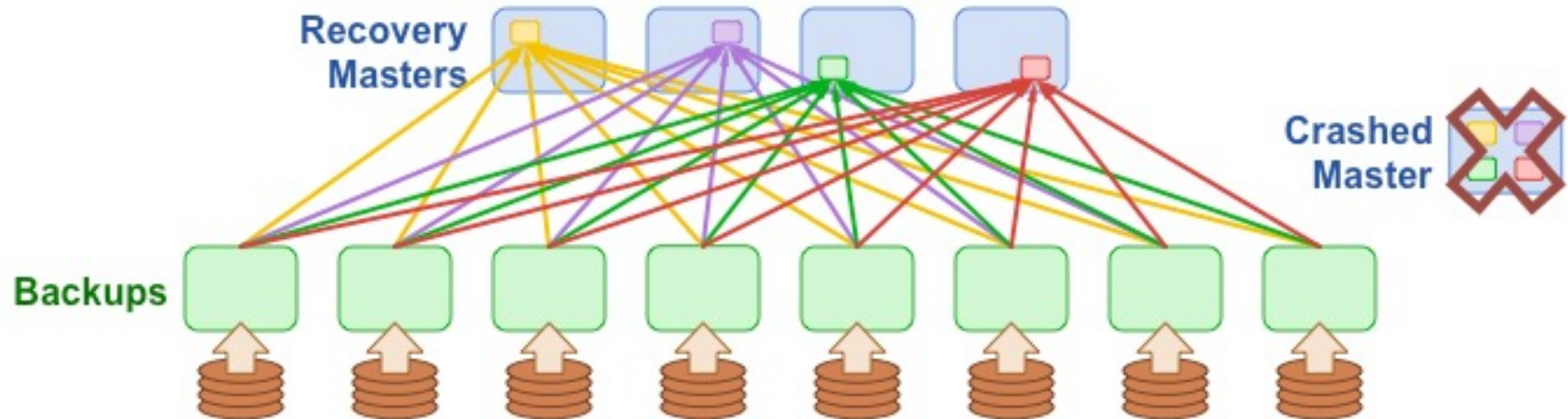
Crash-Recovery: Optimization 1

- Split the state of a server into 1000 parts
 - Recovery master reads logs from 1000 backups in parallel
 - Allows aggregating SSD bandwidth of 1000 SSDs
 - We can now read with 1 TB/s ($1000 * 1 \text{ GB/s}$)
- Problem: network is too slow



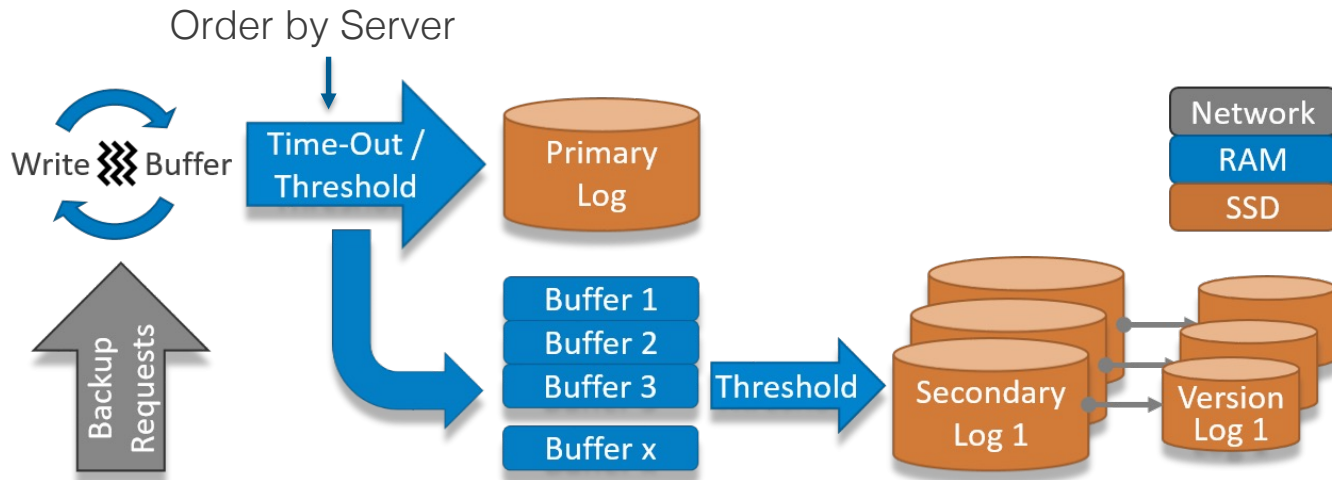
Crash-Recovery: Optimization 2

- Recover the state of a failed server on many servers
 - Allows aggregating network bandwidth



Crash-Recovery: Optimization 3

- Primary Log: stores data chunks on SSD as fast as possible.
- Secondary Logs: **sort data chunks by server to speed up recovery.**
- Version Logs: store version numbers beneath logs for recovery and reorganization.

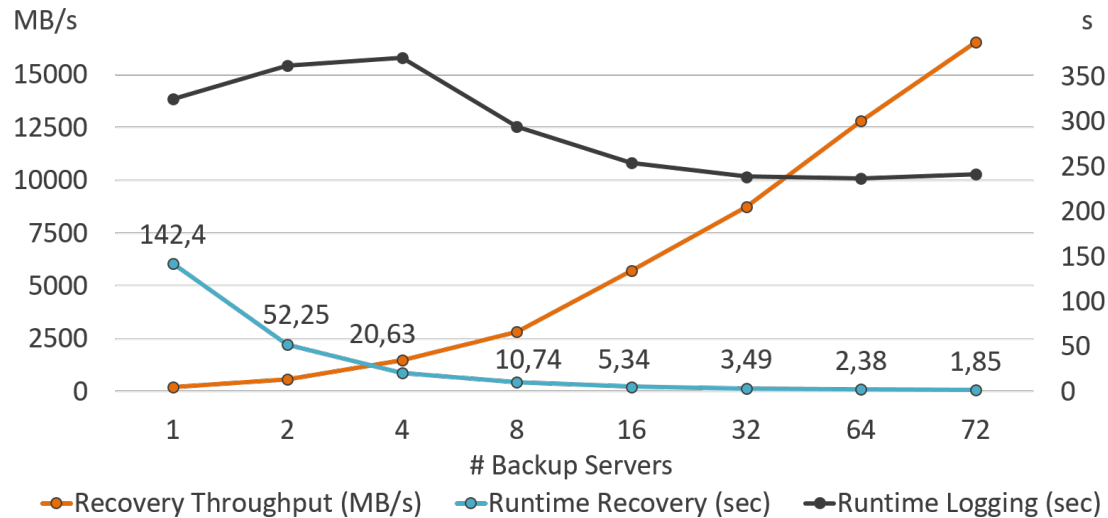


Fast Recovery

- Recovery benchmark:
 - The master server creates 500,000,000 64-byte chunks resulting in more than 30 GB in-memory state
 - The state of one server is scattered using backup zones
 - In this experiment 144 backup zones are replicated on 72 backup servers
 - After logging all chunks, the master is killed which initiates the recovery process for all 144 backup zones.

Fast Recovery

- Results:
 - The recovery times improve with the number of backup servers.
 - With 72 backup servers the complete recovery process took less than 2 s.
 - Recovery throughput >16.5 GB/s.



12. Zusammenfassung

- Es gab und gibt viele verteilte In-Memory Systeme
- DXRAM war ein Projekt an der HHU, 2013 - 2018
 - Weitere Infos:
<https://www.cs.hhu.de/en/lehrstuehle-und-arbeitsgruppen/betriebssysteme/forschung/dxram>